



LAB PRACTICE № 8: STACKS AND QUEUES

COMP1020: OOP, Algorithms, and Data Structures

Week 08

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your classes and methods should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables, methods and classes should have meaningful names, and should be followed Java programming convention.
- Academic honesty is required in all work you submit to be graded. Although you can discuss among yourself at high level how to solve the lab problem, you should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions.
- You should upload your .java file(s) to both the Canvas and CMS Autograder **before the end of the laboratory session** unless the instructor gave a specified deadline.
 - (a) No Canvas submission will get 0 point (of course).
 - (b) **No CMS Autograder submission will get 20% deduction.**
- Submit separate *.java file for each Lab problem with the exact file name specified in each lab question (do not add your student ID or your name). For example: **HelloWorld.java**.
- Late submission of lab practice without an approved extension will incur the following penalties:
 - (a) No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 - (b) The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 - (c) The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.

Learning Outcome

Topics: Stacks and Queues in Java.

Outcome: Acquire the skill of using and creating own stacks and queues in solving computing problems.

CMS Autograder

Direct access to this CMS lab is followed: <https://www.cmsvinuni.online>

Note: Due to the current limitation of the Autograder, you must comment out all the `package` declaration before submission.

Problem 1 - Bracket Sequence

In this problem, the user is requested to input a bracket sequence that only can contain bracket symbols '(', ')', '{', '}', '[', ']', '<', '>'. Next, determine if the input is supported and valid:

Case	Input	Supported?	Valid?
1	{()()())[()]}	YES	YES
2	{()[()	YES	NO
3	<()()())[>	NO	-

- Case 1 & 2 are supported because both cases using the supported bracket symbols.
- Case 2 is invalid because there is no bracket matching in the sequence.
- Case 3 is not supported because of the bracket symbols '<' and '>'. *We do not need to check the validity of a bracket sequence when it is not supported.*

You are supposed to do the following to complete this problem:

- Write a BracketSequence class (in a BracketSequence.java file) with a constructor that accepts one String argument.
- Write two methods: `isSupported()` and `isValid()` to check if the string is supported and if it is a valid bracket sequence.
- Use stack to determine whether it is a valid bracket sequence and print the result according to the input/output format in the examples.
- *Hint:* Use a for-loop to check if it is supported.
- In the `main()` method, repeatedly ask user to enter a bracket sequence and first call `isSupported()` method to check if it is supported, if not, printout as shown in example and EXIT the loop (i.e., terminate your program). If it is supported, check if it is valid and printout result accordingly.

Listing 1: Template for Problem 1.

```
import java.util.*;

public class BracketSequence {
```

```

Enter a bracket sequence: ()
() is a VALID bracket sequence.

Enter a bracket sequence: ([)]
([)] is an INVALID bracket sequence!!!

Enter a bracket sequence: <>
|<> is NOT supported (terminated)!

```

Figure 1: Example A.

```

Enter a bracket sequence: ()([]{})
()([]{}) is a VALID bracket sequence.

Enter a bracket sequence: (({(){})))[[
(({})){}][ is an INVALID bracket sequence!!!

Enter a bracket sequence: exit
exit is NOT supported (terminated)!

```

Figure 2: Example B.

```

private String bracketSeq;

public BracketSequence() {
    bracketSeq = "";
}

public BracketSequence(String s) {
    bracketSeq = s;
}

public boolean isSupported() {
    // TODO: to check if the string is supported
    // ...
}

public boolean isValid() {
    Stack<Character> s = new Stack<Character>();
    boolean ok = true;

    // TODO: to check if it is a valid bracket sequence.
    // ...
    return ok;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String seq;

```

```

int cnt = 0;

while (true) {
    System.out.print("\nEnter a bracket sequence: ");
    seq = sc.nextLine();

    BracketSequence bs = new BracketSequence(seq);

    if (!bs.isSupported()) {
        System.out.println(seq + " is NOT supported (↔
        terminated)!");
        break;
    }

    if (!bs.isValid()) {
        System.out.println(seq + " is an INVALID bracket ↔
        sequence!!!");
    } else {
        System.out.println(seq + " is a VALID bracket sequence↔
        .");
    }

    if (cnt++ > 5) {
        break;
    }
}

sc.close();
}
}

```

Problem 2 - Restaurant

A restaurant has a special way to serve its customers. There is a queue of customers already waiting to buy food. The chef will cook the dishes from the menu one at a time in order, repeating the first dish after the last dish is cooked. When a dish is done, it will get served to the first customer who can afford it. In other words, when a dish is being sold:

- The customer at the front of the queue will take the dish and leave the queue if he or she has enough money.
- If the customer doesn't have enough money then he or she will move to the end of the queue (to wait for the next dish).
- This continue until the dish is bought.
- If everyone has had their chance but no one could buy the dish, it is discarded and the chef will move on to the next dish.

When no more dish can be bought by anyone, the remaining people in the queue will leave the queue (and starve for that day!). For each person, print the dish he or she bought, or announce that he or she couldn't buy any dish in the format seen below.

To complete this problem, you should do the following:

1. Create a `Restaurant.java` file with two classes `Customer` and `Dish` as shown in Listing 1.
2. Create `Restaurant` class with two instant variables `customers` and `dishes`, which are `Queue` type with element are `Customer` and `Dish` objects, respectively.
3. Create a constructor with no argument to initialize `customers` and `dishes`.
4. Create a method called `readDishes()` with one argument of type `Scanner` to prompt the user to enter the number of dishes in the menu, for each dish, prompt user to enter the name and cost. Note that dish name may contain space(s) in between (there is always a space between name and cost). Then create a `Dish` object and add to `dishes` queue.
5. Create a method called `readCustomers()` to perform similar input, where a customer has two properties: `name` and `account balance`. Note that customer name may have space(s) in between (there is always a space between name and account).
6. When someone buy a dish, announce it in the format seen below.
7. After no further dish can be served, announce the people who couldn't buy any meal, in the order that they were standing in the queue.

```
Enter the number of dish(s): 3
Enter dish 1 (<name> <cost>): Bread 10
Enter dish 2 (<name> <cost>): Pho 20
Enter dish 3 (<name> <cost>): Bun Bo Hue 30
Enter the number of waiting customer(s): 4
Enter customer 1 (<name> <account>): Ali Baba 100
Enter customer 2 (<name> <account>): Harry Potter 15
Enter customer 3 (<name> <account>): Mickey 30
Enter customer 4 (<name> <account>): Jean Valjean 5
Ali Baba bought Bread!
Mickey bought Pho!
Harry Potter bought Bread!
Jean Valjean didn't have enough money for any dish!
```

Figure 3: Example A.

Listing 2: Template for Problem 3.

```
import java.util.*;

class Dish {
    String name;
    int cost;

    Dish(String name, int cost) {
```

```

Enter the number of dish(s): 3
Enter dish 1 (<name> <cost>): Bread 10
Enter dish 2 (<name> <cost>): Pho 20
Enter dish 3 (<name> <cost>): Bun Bo Hue 30
Enter the number of waiting customer(s): 4
Enter customer 1 (<name> <account>): Ali Baba 100
Enter customer 2 (<name> <account>): Harry Potter 30
Enter customer 3 (<name> <account>): Mickey 30
Enter customer 4 (<name> <account>): Jean Valjean 50
Ali Baba bought Bread!
Harry Potter bought Pho!
Mickey bought Bun Bo Hue!
Jean Valjean bought Bread!

```

Figure 4: Example B.

```

        this.name = name;
        this.cost = cost;
    }
};

class Customer {
    String name;
    int account;

    Customer(String name, int account) {
        this.name = name;
        this.account = account;
    }
};

public class Restaurant {

    Queue<Dish> dishes;
    Queue<Customer> customers;

    public Restaurant() {
        dishes = new ...;
        customers = new ...;
    }

    public void readDishes(Scanner sc) {
        System.out.printf("Enter the number of dish(s): ");
        ...
    }

    public void readCustomers(Scanner sc) {
        System.out.printf("Enter the number of waiting customer(s): ")↵
        ;
    }
}

```

```
        ...
    }

    public void processOrders() {
        ...
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Restaurant testObj = new Restaurant();

        testObj.readDishes(sc);
        testObj.readCustomers(sc);
        testObj.processOrders();

        sc.close();
    }
}
```
